

.LTA Schema Reference

For Worlds and Models



Version 2002.04.29

Legal Notice

The LithTech™ Development System is copyright 1998-2002 by LithTech, Inc., Kirkland, Washington, U.S.A. All rights reserved.

ImageMagick is copyright 2000 by ImageMagick Studio, a non-profit organization dedicated to making software imaging solutions freely available.

The FreeType Project is copyright 1996-2000 by David Turner, Robert Wilhelm, and Werner Lemberg. All rights reserved except as specified in the *Credits* chapter.

Table of Contents

.LTA Exchange File Format.....	7
Structure	7
Syntax.....	8
Schema	8
Extending the .LTA File Format.....	8
Model Schema .LTA Reference.....	17
Model Schema Node Hierarchy.....	18
add-anim-weightssets.....	20
add-childmodels	21
add-deformer	22
add-node-obb.....	23
add-node-obb-list.....	24
add-sockets.....	25
anim.....	26
anims.....	27
anim-binding	28
anim-bindings	29
anim-weightset.....	30
anim-weightssets.....	31
animset.....	32
appearance	33
child-model.....	34
colors.....	35
col-ifs	36
coord-frame-type	37
diffuse	38
dims	39
dists.....	40
filename	41
frames	42
geometry.....	43
hierarchy	44
influences	45
interp-time.....	46
keyframe	47
lt-model-0.....	48
material.....	49
matrix.....	50
mesh	51
node-relations.....	52
normals	53
nrm-fs.....	54
on-load-cmds	55
orientation	56
parent.....	57
piece-priorities	58
pos	59
posquat	60
quat.....	61
render-priority	62
set-global-radius.....	63
set-node-flags	64
set-repl-lod-original	65
shape.....	66
size	67

Jupiter System

shininess	68
skel-deformer	69
socket	70
specular	71
target	72
texture-index	73
tex-fs	74
texture-bindings	75
tools-info	76
transform	77
translation	78
tri-%	79
tri-fs	80
uvs	81
vertex	82
weights	83
weight-set	84
weights	85
World Schema .LTA Reference	86
World Schema .LTA Node Hierarchy	87
World Schema Example	89
bool	93
childlist	94
color	95
color	96
data	97
description	98
dist	99
editpoly	100
f	101
flags	102
forward	103
header	104
infostring	105
label	106
longint	107
lookat	108
lookatdist	109
n	110
name	111
nodeid	112
navigator	113
navigatorpositem	114
navigatorposlist	115
physicsmaterial	116
pointlist	117
polyhedron	118
polyhedronlist	119
polylist	120
position	121
properties	122
propid	123
proplist	124
real	125
right	126
rotation	127
shade	128

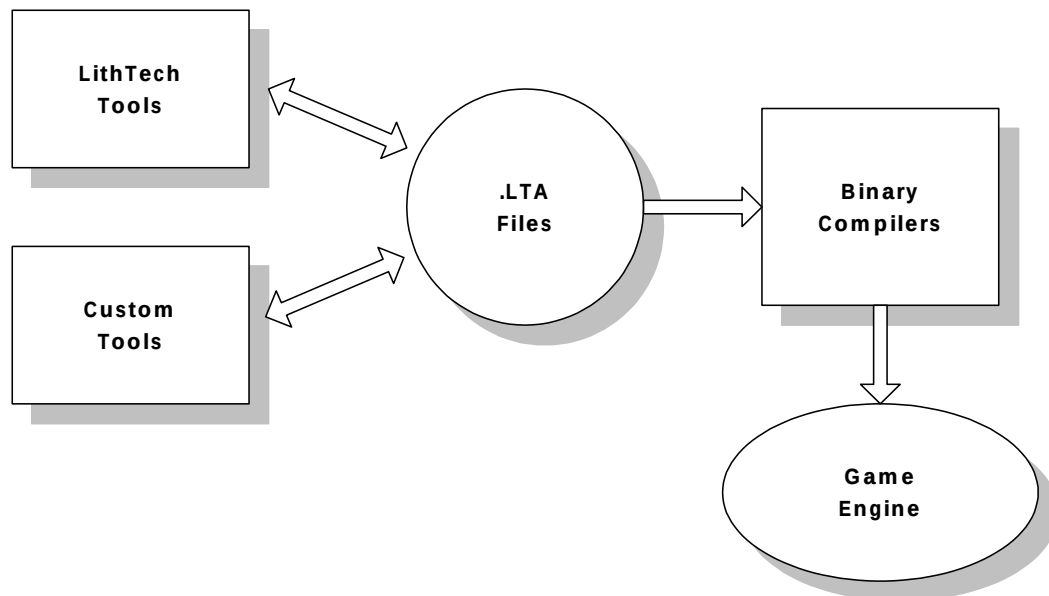
.LTA Schema Reference

sticktopoly.....	129
string.....	130
textureinfo.....	131
type.....	132
up.....	133
vector.....	134
versioncode.....	135
world.....	136
worldnode.....	137

.LTA Exchange File Format

The .LTA (LithTech ASCII) exchange file format provides a format for describing structured data. It enables precise communication of information between the LithTech tools (DEdit, ModelEdit) and the data packers for conversion to platform specific binary files. LithTech uses the .LTA file format as the description format of worlds and models that are used by the tools. The .LTA files are then translated to a platform specific binary format for the engine.

Note: The .LTA format can be compressed into the LithTech Compressed format (.LTC). To compress a file into an .LTC file in your code, set the **SaveNode** function's **bCompress** parameter to true as explained in the [LTA Library](#) section below.



By extending the .LTA format, game developers can define an unlimited set of customized nodes for their own communication needs. They can extend the current tools or create their own. An .LTA node can declare its associated data to be a spatial coordinate, the name of a weapon, a skin's identification, the shininess of an object, geometric information about a model, or any other desired data.

LithTech includes an extensive .LTA reference for both the Model and World schemata.

Structure

Fundamentally, the .LTA format is a collection of lists delineated by parentheses. Values are the members of these lists. The collection can be organized hierarchically by having one collection be a member of another collection.

The format's simplicity makes it easy to parse, and its inherent hierarchy makes it easy to transform into internal data structures.

An .LTA file is composed of a nested list of parenthetical nodes. The following example shows an .LTA file consisting of one top-level node and several member nodes.

```
(  
  (  
    (  
    )  
  )  
  (  
  )  
)
```

The example is presented in an indented fashion for ease of reading. The .LTA parser ignores the indentation and extra spaces. It is the parentheses that define the hierarchical structure. The .LTA files do not need to be formatted using indentation and extra white spaces. Instead, all nodes in a file can be on one line if desired as shown below:

```
((()())())
```

An .LTA file can contain a nested list of nodes of substantial depth and complexity.

Syntax

The .LTA format recognizes all characters. Reserved characters include:

- parentheses
- quotation marks
- spaces

Parentheses delineate the hierarchical structure of the list. Quotation marks contain string values. Spaces delimit values. All other characters are free for use in strings, types, and values.

There are no escape characters for the reserved characters. Using the reserved characters in a value or type will break the .LTA file.

The .LTA format is case-sensitive.

All values in an .LTA file are treated as strings. Values with quotes around them are tagged by the parser as being true strings. The parser builds a parse tree, which retains all the values as strings with quoted strings tagged.

Node

An .LTA node consists of an open parenthesis, a close parenthesis, and the information between the parentheses.

Each node may contain any type of ASCII data, such as alphanumeric values, strings, and also other nodes. The following example node contains four separate datum:

```
( superhero "Nigel" 934.97 ! )
```

A node that contains another node is called a list node (it is also a parent node). The inner node is the child node, as shown below.

```
(list or parent node  
  (child)  
)
```

Schema

The .LTA format does not define any default types or tags. Only the hierarchical structuring tags are defined by the .LTA format. The .LTA format can be extended to create .LTA schema. An .LTA schema is a description of an entity using the .LTA format. Each schema will have its own "types" of nodes, permissible values, and hierarchy.

For examples of schemata, see the Model Schema LTA Reference and World Schema LTA Reference chapters of this document. The therein are used by the ModelEdit and DEdit tools, respectively, to communicate model and world data between the tools and the LithTech compilers.

Extending the .LTA File Format

You can extend the .LTA exchange file format by creating a new schema and an associated parser (reader and writer). After the schema has been designed, the .LTA reader and writer can be coded to properly parse it. If you alter the schema after the parser has been coded, you will have to update the reader for the new schema. The reader can check for older format types, and provide a warning. As long as the writer no longer writes to the old format, one format can be migrated into another.

Designing .LTA Schemata

The .LTA schemata define nodes, symbols, and hierarchies. The format provides freedom on how to design a schema. Some suggestions are given below. There are, of course, many different ways to design a schema.

When designing an .LTA schema, keep in mind containment and hierarchical relationships. The .LTA nodes are their own scope. Named nodes within other nodes are unique. For example,
(foo (bif foo))

.LTA Schema Reference

Both foo can be treated differently since the second foo belongs in the bif scope and the first foo belongs in its parent scope (which in this case is the global scope).

Nodes contain symbols and other nodes. Symbols can be used to define the type of the node. For example, if a node will contain data about a superhero, the first symbol of the node can define the “type” of node.

```
(superhero)
```

Symbols can also contain strings. Strings can be used to identify a specific instance of a node. For example, the second symbol in a node can be a string identifying the specific node.

```
(hero-name "Sunburst")
```

When designing the node hierarchy, logical relationships should be devised. For example, if a node sets the powers of a superhero, then the powers node can be a child of the superhero node.

```
(superhero
  (powers
    (flight passive body)
  )
)
```

Nodes can contain any number of symbols. While some symbols might define the type of the node, and others might be strings identifying the node, at some point the value of the node will need to be set. One way to set a value is to use a second symbol in the node. The example below sets the superhero's primary stats:

```
(superhero
  (primary-stats
    (strength 12)
    (agility 30)
    (vitality 17)
    (intellect 14)
    (will 10)
    (charm 18)
  )
)
```

Symbols might also be optional. The parse logic of an extended .LTA reader might use the exclamation point as a flag to indicate that the node is in an “enraged” mode. The reader can define a set of node types that allow the flag.

```
(superhero !
  (hero-name "Sunburst")
)
```

For ease of reading, explicit nodes are best. For example, to explicitly define a superhero's attributes, the following node hierarchy might be designed:

```
(superhero
  (hero-name "Sunburst")
  (primary-stats
    (strength 12)
    (agility 30)
    (vitality 17)
    (intellect 14)
    (will 10)
    (charm 18)
  )
  (enraged yes)
)
```

However, to improve processing performance, a more implicit design could be used. Here, the .LTA reader expects the first symbol to be the type. For superhero types, it expects the next symbol to be name, followed by the six stats in a specific order. If it finds an exclamation point in the node, it knows that the superhero is enraged.

```
( superhero "Sunburst" 12 30 17 14 10 18 ! )
```

Obviously, there are numerous design options. For examples of specific LTA schema implementations, see the reference chapters of this document.

Sample Schema

The following example defines the superhero format for a superhero CRPG game. An .LTA file is used to describe the capabilities for a superhero character. This file is used by a tool to create characters, save them into the .LTA file and then read it back in for further editing or for use by the game. The top .LTA node is the node "superhero". This node is the global node for describing superhero type data.

```
(superhero
  (hero-name "Sunburst")
  (level 2)
  (primary-stats
    (strength 12)
    (agility 30)
    (vitality 17)
    (intellect 14)
    (will 10)
    (charm 18))
  (secondday-stats
    (wound-points 14 )
    (initiative 4)
    (will-resist 6))
  (powers
    (flight passive body)
    (fire passive body)
    (fire beam hands))
  (skills
    (bluff 8)
    (diplomacy 5)
    (knowledge "physics")
    (dodge 2))
  (biography
    (file-name "sunburstbio.txt"))
)
```

The superhero node contains seven other nodes. These nodes in turn contain their own data. The order of the superhero subnodes does not matter.

If a superhero node is missing a subnode, the reader can either fail or create a suitable default value for the missing subnode.

If there is a subnode that is not understood by the reader it can simply ignore that node.

Designing .LTA Readers and Writers

Once the file is in memory it must be evaluated to get the data out of it. A Reader must be written to evaluate the parse tree. A Writer must be made to translate the internal data into the .LTA file by creating a parse tree that is saved to a file.

For the superhero example, a Reader must be created to convert the .LTA file into our internal data format. A Writer is also needed to turn that format into an .LTA file.

The Reader uses the .LTA API (located in the \engine\tools\libs\ltamgr directory) to transform the parse tree into an internal data format. The internal data format for the superhero is the **SSuperHero** structure.

```
struct SSuperHero {
  struct SPrimaryStats {
    int str, agil, vit, iq, will, charm ;
  };
};
```

```
string name ;
SPPrimaryStats m_stats ;
int m_WndPnts;

...
};
```

The following function takes in a node that has as its first member the key word "superhero". The function picks out the relevant data from the partial parse tree and moves the data into the new_superhero data structure.

```
void parse_superhero( CLTANode *root, SSuperHero & new_superhero )
{
    // zero out superhero data struct.
    init( new_superhero );

    if( root->GetElement(0)->value == "superhero" )
    {
        CLTANode *name = CLTAUtil::ShallFindList(root, "hero-name" ) ;
        if( name )
            new_superhero.m_name = name->GetValue() ;

        CLTANode *prim_stats = CLTAUtil::ShallowFindList( root, "primary-stats");
        if( prim_stats != NULL )
        {
            // we've found the primary stats, convert the string to
            // integer.
            new_superhero.m_stas.str = atoi(CLTAUtil::ShallowFindList ( prim_stats, "strength"
)->GetElement(1)->GetValue());

            new_superhero.m_stas.agl = atoi(CLTAUtil::ShallowFindList ( prim_stats, "agility" )-
>GetElement(1)->GetValue());
            // and so on.
        }
    }
}
```

In the reader code the file is opened, all the superhero nodes are found and processed. This reader accepts more than one superhero node in the .LTA file.s

```
CLTANode *parse_tree_root ;
vector<CLTANode*> superhero_nodes ;
vector<SSuperHero> superhero_list ;

// create the parse tree
parse_tree_root = CLTANodeReader::LoadEntireFile( "somefile.lta", bIsCompressed );

// this will give us a list of all the nodes that contain superhero in the file.
CLTAUtil::FindAll( superhero_nodes, parse_tree_root, "superhero" );

// resize the superhero list to the number of superhero nodes in the parse tree.
superhero_list.resize( superhero_nodes.size() );

// parse all the superhero nodes.
for ( int I = 0 ; I < superhero_nodes.size() ; I++ )
{
    parse_superhero( superhero_nodes[I], superhero_list[I] );
}
```

This method allows for missing nodes, and also nodes not understood by the current parser. If the primary stats node is missing strength, the parse node that points to it is null. The reader program finds that that the strength node is missing and recovers by signaling an error or setting a default strength value of '10' in the c structure.

Nodes need not be in a specified order. If a new "feats" node is added, for example, for which there is no reader support, the parser ignores the node. To create a reader that keeps unknown nodes around, the parse tree has to be preserved in memory, altered nodes reinserted and unneeded nodes deleted from the tree.

The following function examines the data in superhero data structure and creates a parse tree out of it.

```
void write_superhero( CLTANodeBuilder & parse_tree, SsuperHero & superhero )
{
    parse_tree.Push("superhero");
    parse_tree.Push("hero-name");
    parse_tree.AddValue( superhero.m_name );
    parse_tree.Pop();
    parse_tree.Push("level");
    parse_tree.AddValue( superhero.m_level );
    parse_tree.Pop();
    ...
    parse_tree.Pop();
}
```

The following function is the .LTA writer for the superhero example. The function above creates the parse tree and this function uses the **CLTANodeWriter** object on it.

```
void write_superhero_LTA_file( char *filename, list<SSuperHero> & super_list )
{
    CLTAWriter writer("somefile.lta", false);
    if( writer.IsValid() )
    {
        for( int I = 0 ; I < super_list.size() ; I++ )
        {
            CLTANodeBuilder list_builder ;
            write_superhero( list_builder, super_list[I]);
            CLTANodeWriter::SaveNode( list_builder.getRoot(), &writer );
        }
        out_file.close();
    }
    else
    {
        // error opening file...
    }
}
```

Small Files

Small files can be kept in memory and interpreted in memory. Files like the superhero data files can all be kept in memory until used.

Very Large Files

Large files, ones that are larger than one meg on disk should be evaluated as its being read in. Larger files need to be read in pieces, and one unit is read in that unit is truncated from the tree.

The .LTA Library

The .LTA parse library contains classes and functions that parse and write .LTA files. The classes and functions are defined in the header files located in the \engine\tools\libs\ltamgr directory.

There are several classes that provide .LTA operations:

CLTANode Provides the basic data type of the parse tree. Each element in the parse tree is represented by a **CLTANode** instance.

CLTANodeBuilder Programmatically builds a parse tree using stack-like functions.

CLTARReader Provides functions to read an .LTA file. Handles tokenization of the .LTA file.

CLTANodeReader Provides utility functions for loading entire parse trees or parts of parse trees..

CLTAWriter Provides functions to write to an .LTA file. The output stream for a parse tree.

CLTANodeWriter Provides utility functions for writing entire parse trees.

CLTAUtil Provides several miscellaneous utility functions for accessing lists.

The Parse Tree

The parse tree is the in-memory representation of an .LTA file. The .LTA parser constructs this tree as the file is read. It is useful to understand the in-memory structure of the .LTA file so that you can understand how the reader works. The most basic reader steps and searches through this tree to find needed information.

Given an ASCII .LTA file looks like this:

```
(foo (bar (a b) 2.0))
```

Then in memory the parse node structure looks like this :

```
lta_node->children[0]->value = "foo"
      ->children[1]->children[0] = "bar"
->children[1]->children[1]->children[0] = "a"
->children[1]->children[1]->children[1] = "b"
->children[1]->children[0] = "2.0"
```

Notice that the top node is a container. Its first member is an atom that contains the value foo. The second child of that container is another container. This container has three children with the values foo, container, and 2.0.

All values are held at the leaves of this parse tree. These values are still strings when the reader gets the parse tree.

CLTANode

This class represents the nodes of an .LTA parse tree. As the .LTA file is read into memory, a parse tree is constructed. Each node in the tree is a **CLTANode** instance.

CLTANode data members are all private for portability. All values for the data members are strings.

The **CLTANodes** represent lists, atoms, and nils. A list is a node that has children. An atom is a node that contains one or more symbols but no children (a leaf node). A nil is a node that has no children and contains no symbols.

This class is defined in the LTANode.h header file and exposes the following functions:

AllocateElements Specify the number of child nodes for this node. Each child node must be set prior to using this node.

AppendElement Append a **CLTANode** pointer to the list of child pointers for this node. This method is used in the .LTA reader when the parser is filling in a list node. This should not be used frequently since it requires the resizing of the internal arrays, and can cause a large performance hit. Instead, the node should call **AllocateElements** with the proper size, and then set each element.

CLTANode() Construct a new atom node with no value (a nil).

CLTANode (uint32 nNumElements) Construct a list node. The **nNumElements** parameter sets the number of child nodes.

CLTANode(const char* pszValue, bool bString) Construct a new atom node. The **pszValue** parameter is the value. The **bString** parameter indicates whether the internal type is a true string.

Free Remove and frees all descendents of this node.

GetElement Return the parse node at a given index (0 based). This will return null if there is no element at the index. This should only be used on lists.

GetValue Return a pointer to the value of this **CLTANode**. This can be converted to a number if a number is expected. The value does not contain quotation marks, even if it is a true string. Use **IsString** to determine whether or not the value is a true string.

IsAtom Return true if the node has no children.

IsList Return true if the node has children.

IsString Return true if the parse node's value is a string. A string is a value that was originally contained in quotes. The **CLTANode**'s value does not contain quotes in either case.

GetNumElements Return the number of children this node contains.

SetElement Set the value of a specified child node of this list node.

SetValue Set the value of this **CLTANode**.

CLTANodeBuilder class

This class builds .LTA parse trees. It is defined in the LTANodeBuilder.h header file and exposes the following functions:

AbortBuild Abort the build process. Deletes all nodes that have already been created.

AddElement Add an element to the current node on the stack. This assumes that the **pElement** parameter was created with **new**. The new element instance is orphaned by the caller, meaning that any pointers to the instance are invalid after this call.

AddValue Add a value node as a child to the list that was most recently pushed onto the stack. The four versions of this function support double, Boolean, integer, and string values.

DetachHead Detach the parse tree currently being built. This pops all nodes off the stack and clears all memory associated with the builder. This should be called only when the tree building process is completed. It is the only method of obtaining the root element of the tree. The returned node must be deleted by the caller.

Free Free all memory associated with the **CLTANodeBuilder** instance. Use this when memory is critical and the program cannot wait until the instance is destroyed.

GetDepth Return the current depth of the stack.

Init Initialize the builder. Should be used between building two parse trees to reset the builder.

Pop Remove the last element pushed onto the stack and create a list node instance. This new list node instance is added to the list of children of the last **CLTANode** instance pushed on to the stack.

PopAll Remove all nodes from the stack. This ends the build process.

Push Push a new node onto the parse tree stack. This is similar to the opening parenthesis in an .LTA file. This will cause a list node object to be created when the new node is popped off the stack. It will contain all the node elements created after the push, until **Push** is called again.

Push(const char* pszValue, bool bString) Another version of the Push function. In addition to doing the push a value (**pszValue**) is added to the list as the first element.

CLTANodeReader

Provides utility functions for loading entire parse trees or parts of parse trees. This class is defined in the LTANodeReader.h header file and exposes the following static functions:

LoadEntireFile Open the specified file and load the entire file into a node tree.

LoadNode Parse through the current tree until a value matching **pszStartString** is found. Then create a root node and add the value to it. It will then parse through the file until the list is properly closed. This can be used to pull certain sections of a file out. Note that this is sequential, and if you want to back up, the file should be closed and reopened. Another thing to note is that this value must come after an open parenthesis. So, (a, b) with **LoadNode("b")** would return NULL, since the b does not come after the open parenthesis.

CLTANodeWriter

Provides utility functions for writing entire parse trees. This class is defined in the LTANodeWriter.h header file and exposes the following static functions:

SaveNode(CLTANode* pNode, const char* pszFilename, bool bCompress) Create an .LTA file (pszFileName) from a given a node (pNode) to start at. If bCompress is set to true, the file will be saved in .LTC format.

SaveNode(CLTANode* pNode, CLTAWriter* pWriter) Add the tree to the writer (pWriter) given a node (pNode) to start at.

CLTARader

Provides functions to read a .LTA file. Handles tokenization of the .LTA file. This class is defined in the LTARader.h header file and exposes the following functions:

Open Open a specific file for reading.

Close Close the file that is currently open.

IsValid Determine if the file is valid for reading.

NextToken Return the next token type and value from the file.

CLTAWriter

Provides functions to write to an .LTA file. The output stream for a parse tree. This class is defined in the LTAWriter.h header file and exposes the following functions:

Open Open a specific file for reading.

Close Close the file that is currently open.

IsValid Determine if the file is valid for reading.

BeginNode Mark the beginning of a new node. This should be called whenever writing out a new list. It allows for internal formatting so that output appears correctly.

EndNode Mark the end of a list. See **BeginNode**.

Write Write string values to the file.

CLTAUtil

CLTAUtil is a collection of odds and ends functions that can aid in the traversal of lists. It contains helper functions, all static, to determine information about files, and perform searching on lists.

IsFileCompressed Given a filename, this determines if the file is compressed or not.

GetMemoryUsage Given a node, this determines the amount of memory being used in bytes by the node and all of its children

ShallowFindList Searches through a node and looks for a list that has the first element set to the specified value. Very useful for finding keys in a list of children.

FindList Searches through a node and looks for the first list to have an element as the first item with a string that matches the specified value. This will search into the tree recursively, meaning that all the children of the passed node will be searched, then all of its children, and so on.

FindAll Builds up a list of all the lists that have the first element set to the specified value. Returns the number of elements added below the specified start node

An .LTA Sample Code

This program opens up a model .LTA file, searches for the influence (member of skel-deform) node and prints out its contents.

```
void main(int argc, char **argv)
{
    CLTARader Reader(argv[1], CLTAUtil::IsCompressed(argv[1]));
    ofstream fresult("c:\\LT\\tmp.txt");
    vector<CLTANode*> allAnims ;
    CLTANode* pFileRoot ;
```

```
pFileRoot = CLTANodeReader::LoadEntireTree(&Reader);

CLTAUtil::FindAll(allAnims,pFileRoot,"influences");

fresult<< "--- > influences " << argv[1] << endl;
if(allAnims.size() == 0 )
    fresult << " no anims ... " << endl;
for( int i = 0 ;i < allAnims.size() ;i++ )
{
    CLTANode *list_items = allAnims[i]->GetElement(1) ;

    for( int i = 0 ; i < list_items->GetNumElements() ;i++ )
    {
        fresult << list_items->GetElement( i )->GetValue() << endl;
    }
}
fresult << " *** " << endl;
fresult.close();
}
```

Model Schema .LTA Reference

This reference describes the nodes used for the LithTech model database. A model is a collection of basic geometry and animation information, plus a collection of modifiers that are applied when the model is loaded.

The nodes are presented in alphabetical order, and in a standard format described below.

Model Schema Node Hierarchy

The following structure lists all the various nodes in the model database. This structure only lists typed nodes, see the individual node references for member node information.

```
(lt-model-0
  (coord-frame-type)
  (hierarchy
    (children
      (transform
        (matrix)
        (coord-frame-type)
        (children)
      )
      (shape
        (parent)
        (geometry)
        (appearance)
      )
    )
  )
  (shape
    (parent)
    (geometry
      (mesh
        (coord-frame-type)
        (vertex)
        (uvs)
        (normals)
        (colors)
        (tri-fs)
        (nrm-fs)
        (tex-fs)
        (col-fs)
      )
    )
    (appearance
      (material
        (texture-index)
        (diffuse)
        (ambient)
        (emissive)
        (specular)
        (shininess)
      )
    )
    (render-priority)
  )
  (animset
    (keyframe
      (keyframe)
    )
    (anims)
  )
  (on-load-cmds
    (add-deformer
      (skel-deformer
        (target)
        (influences)
        (weightsets)
      )
    )
    (add-node-obb-list
      (add-node-obb
        (pos)
        (orientation)
```

```
        (size)
      )
    )
  (add-sockets
    (socket
      (parent)
      (pos)
      (quat)
    )
  )
  (set-global-radius)
  (add-childmodels)
  (add-anim-weightsets
    (
      (anim-weightset
        (name)
        (weights)
      )
    )
  )
  (anim-bindings
    (
      (anim-binding
        (name)
        (dims)
        (translation)
        (weight-set)
      )
    )
  )
  (set-node-flags)
  (set-repl-lod-original
    (dists)
    (piece-properties)
    (tri-%)
  )
  (child-model
    (filename)
    (node-relations)
  )
)
(tools-info
  (
    (texture-binding)
  )
)
```

add-anim-weightsets

Adds **anim-weightset** nodes to the model.

```
(add-anim-weightsets
  (
    (anim-weightset)
    ...
  )
)
```

Parent nodes on-load-cmds

Number of occurrences per parent node 0 or 1

Required member nodes A collection of 0 or more **anim-weightset** nodes

Optional member nodes None

Default Value None

add-childmodels

Adds **child-models** to model database.

```
(add-childmodels
  (
    (child-model)
    (child-model)
  )
)
```

Parent nodes on-load-cmds

Number of occurrences per parent node 0 or 1

Required member nodes A collection of 0 or more **childmodel** nodes

Optional member nodes None

Default Value None

add-deformer

Adds deformer to the model database.

Currently, the only deformer available is the **skel-deformer**.

```
(add-deformer
  (
    (skel-deformer)
  )
)
```

Parent nodes on-load-cmds

Number of occurrences per parent node 0 or more

Required member nodes A collection of 0 or more **skel-deformer** nodes

Optional member nodes None

Default Value None

add-node-obb

Contains information about an oriented bounding box (OBB) for the model.

```
(add-node-obb "node-name"  
  ( pos )  
  ( orientation )  
  ( size )  
)
```

Parent nodes add-node-obb-list

Number of occurrences per parent node 0 or more

Required member nodes None

Optional member nodes pos, orientation, and size

Default Value None

add-node-obbb-list

Adds a list of **add-node-obbb** nodes to model database.

```
(add-node-obbb-list  
  ( add-node-obbb )  
  ( add-node-obbb )  
  ( ... )  
)
```

Parent nodes on-load-cmds

Number of occurrences per parent node 0 or more

Required member nodes A collection of 0 or more **add-node-obbb** nodes

Optional member nodes None

Default Value None

add-sockets

Adds a collection of **socket** nodes to model database.

```
(add-socket
  (
    (socket)
  )
)
```

Parent nodes on-load-cmds

Number of occurrences per parent node 0 or more

Required member nodes A collection of 0 or more **socket** nodes

Optional member nodes None

Default Value None

anim

Contains data that modifies a **transform** or **mesh** node.

The **target** member must be a previously defined node in the model database.

If the **target** member node identifies a **mesh** node, the **frames** node must modify an appropriate member node of **mesh**. Currently, only the **vertex** mesh member is modifiable using **anim**.

If the **target** member identifies a **transform**, the **frames** node will modify the transform's position and quaternion values.

The number of fields in the **frames** node must be equal to the number of values in the times member of **keyframe** node in the **anim** node or its parent **animset** node.

```
(anim "identifier"  
  (target)  
  (keyframe)  
  (frames)  
)
```

Parent nodes anims

Number of occurrences per parent node 0 or more

Required member nodes frames, keyframe, target

Optional member nodes None

Default Value None

anim

Contains a list of **anim** identifiers or a collection of **anim** nodes.

```
(anims
  ("anim-identifier" "anim-identifier")
)
```

```
(anim
  (
    (anim)
    (anim)
  )
)
```

Parent nodes animset

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes 0 or more **anim** nodes

Default Value None

anim-binding

An anim-binding node adds information to an already loaded **animset**. *Translation* is the offset added to the animation's base transform, *dims* is the collision axis aligned bounding box for the associated animation. *Weight-set* is the name of the default weight set to use for the associated animation.

```
(anim-binding
  (name)
  (dims)
  (translation)
  (weight-set)
)
```

Parent nodes anim-bindings

Number of occurrences per parent node One or more

Required member nodes dims, name, translations, weight-set, interp-time

Optional member nodes None

Default Value None

anim-bindings

Adds one or more **anim-binding** nodes to model database.

```
(anim-bindings
  (
    (anim-binding)
    (anim-binding)
  )
)
```

Parent nodes on-load-cmds

Number of occurrences per parent node 0 or 1

Required member nodes A collection of 0 or more **anim-binding** nodes

Optional member nodes None

Default Value None

anim-weightset

Contains information about animation weights. The *weights* field is a list of float values for every transform in the model's **hierarchy** node.

```
(anim-weightset
  (name "identifier")
    (weights
      (weight1 weight2 ...)
    )
)
```

Parent nodes add-anim-weightsets

Number of occurrences per parent node 0 or more

Required member nodes weights

Optional member nodes None

Default Value None

anim-weightsets

Adds zero or more **anim-weightset** nodes to model database

```
(anim-weightsets
  (
    (weighsets)
    (weightsets)
  )
)
```

Parent nodes on-load-cmds

Number of occurrences per parent node 0 or more

Required member nodes None

Optional member nodes A collection of 0 or more **weightset** nodes.

Default Value None

animset

Represents a named animation that contains a set of **anim** nodes related to the model database.

```
(animset "identifier"  
  (keyframe)  
  (anims)  
)
```

Parent nodes lt-model-0

Number of occurrences per parent node 0 or more

Required member nodes 0 or more anims nodes

Optional member nodes keyframe

Default Value None

appearance

The appearance node holds the material properties of the associated **shape**.

```
(appearance
  (material)
)
```

Parent nodes shape

Number of occurrences per parent node 0 or more

Required member nodes material

Optional member nodes None

Default Value None

child-model

This node loads another model file into the currently loaded file. **child-model** node allows the current model to access the animations from the “child” model. Through this mechanism many model databases can share one set of animations.

The node-relation node under **child-model** contains a list of offsets to be applied to the animation when calculating the new animation for the model database.

```
(child-model
  (filename)
  (node-relations)
)
```

Parent nodes add-childmodels

Number of occurrences per parent node 0 or more

Required member nodes filename, node-relations

Optional member nodes None

Default Value None

colors

Member of mesh node containing vertex color information. Each vertex color must be specified in float values between zero and one; red, green, blue and alpha components must be supplied, a value of one for alpha means the vertex will be opaque.

```
(colors
  (
    ( red_value green_value blue_value alpha_value )
    ...
  )
)
```

Parent nodes mesh

Number of occurrences per parent node 0 or more

Required member nodes A collection of list nodes containing red, green, blue, and optional alpha values

Optional member nodes None

Default Value None

col-ifs

Contains a list of color index face sets. Each color face set is a list of polygons identified as a series of indices into the colors node.

```
(col-ifs
  ( triangle1_value1 triangle1_value2 triangle1_value3 triangle2_value1 triangle2_value2
    triangle2_value3 ... )
)
```

Parent nodes mesh

Number of occurrences per parent node 1

Required member nodes A list of triangle values in groups of three

Optional member nodes None

Default Value None

coord-frame-type

Sets the coordinate frame system for the parent node.

If **lt-model-0** has a **coord-frame-type** node, it will override all other **coord-frame-type** nodes in the model.

The default value for this node is left-hand, y-up, global.

The possible values for the flags include:

Flag	Possible values
<i>flag1</i>	right-hand, left-hand
<i>flag2</i>	x-up, y-up, z-up
<i>flag3</i>	local, global

```
(coord-frame-type flag1 flag2 flag3 )  
)
```

Parent nodes anim, lt-model-0, mesh, transform

Number of occurrences per parent node 0 or 1

Required member nodes None

Optional member nodes None

Default Value None

diffuse

Specifies the diffuse color component of a material node. Alpha is optional, defaults to one.

```
(diffuse  
  ( r g b [a] )  
)
```

Parent nodes material

Number of occurrences per parent node 0 or 1

Required member nodes A list node containing red, green, blue (and optionally alpha) values.

Optional member nodes None

Default Value None

dims

Defines the axis aligned collision bounding box for an animation.

```
(dims  
  ( x y z )  
)
```

Parent nodes anim-binding

Number of occurrences per parent node 0 or 1

Required member nodes A list node containing the dimensions for the x, y, and z axes

Optional member nodes None

Default Value x = 16, y = 16, z = 128

dists

A list of distances used to switch level of detail geometry. Must contain positive values. This applies to the entire model database.

The number of values in the list must match the number of values in the **tri-%** node of set-**repl-lod-riginal** node.

```
(dists
  (distance1 distance2 distance3 ...)
)
```

Parent nodes set-repl-lod-original

Number of occurrences per parent node 1

Required member nodes A list node containing the distances. The number of distances must match the number of percentages in the **tri-%** node.

Optional member nodes None

Default Value None

filename

Identifies a file for the **child-model** node.

The filename property is a string identifying the relative path from the load directory to the desired file. For example, “../resource/lta/tankmodel”. The .lta extension is not needed.

(filename “*filename*”)

Parent nodes child-model

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes None

Default Value None

frames

Contains vertex or position and quaternion information for each key frame in an animation.

```
(frames
  (posquat ( ... ))
)
```

```
(frames
(vertex ( ... ))
)
```

Parent nodes anim

Number of occurrences per parent node 1

Required member nodes One posquat or one vertex node

Optional member nodes None

Default Value None

geometry

Describes the geometry of the shape.

```
(geometry  
  (mesh)  
)
```

Parent nodes shape

Number of occurrences per parent node 1

Required member nodes mesh

Optional member nodes None

Default Value None

hierarchy

Defines the transformation hierarchy of the model database. Its only member node is **children**.

If the **lt-model-0** node has no identifier, it will default to the hierarchy's identifier.

```
(hierarchy "identifier"  
  (children)  
)
```

Parent nodes lt-model-0

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes 0 or more children nodes

Default Value None

influences

Describes the transforms used by the **skel-deformer** to deform associated **shape**.

```
(influences
  (transform_identifier transform_identifier)
)
```

Parent nodes skel-deformer

Number of occurrences per parent node 1

Required member nodes A list of **transform** identifiers

Optional member nodes None

Default Value None

interp-time

Sets the transition time in milliseconds between concurrent animations.

(interp-time *value*)

Parent nodes anim-binding

Number of occurrences per parent node

Required member nodes

Optional member nodes None

Default Value 200

keyframe

The keyframe node has overloaded meaning: In **animset** it is a node that holds an other **keyframe** node describing the actual time value pairs for the associated **animset**. A **keyframe** node in an **anim** node directly describes the times for the **frame** node's values.

The **keyframe** node contains a list of times and values of equal length. The **values** list is a collection strings used by the engine to uniquely identify each frame of an animation.

The **times** node is a collection of positive ascending numbers in milliseconds that represents a place in time that corresponds to a value in the **frame** node.

In **animset** :

```
(keyframe
  (keyframe "identifier"
    (times ( ... ))
    (values ( ... ))
  )
)
```

In **anim** :

```
(keyframe "identifier"
  (times ( ... ))
)
```

Parent nodes animset, anim

Number of occurrences per parent node 1

Required member nodes times

Optional member nodes None

Default Value For anim, inherit the animset value

lt-model-0

Describes a model database and its modifiers.

Each Model Schema .LTA file must have one (and only one) **lt-model-0** node.

```
(lt-model-0 "identifier"  
  (hierarchy)  
  (coord-frame-type)  
  (shape)  
  (shape)  
  (animset)  
  (animset)  
  (on-load-cmds)  
)
```

Parent nodes This is the top node for the Model Schema.

Number of occurrences per file 1

Required member nodes None

Optional member nodes animset, coord-frame-type, hierarchy, on-load-cmds, shape, tools-info

Default Value None

material

Describes how light effects shapes.

```
(material "identifier"  
  (texture-index)  
  (ambient)  
  (emissive)  
  (specular)  
  (diffuse)  
  (shininess)  
  (texture-index)  
)
```

Parent nodes appearance

Number of occurrences per parent node 1

Required member nodes None

Required member nodes None

Optional member nodes ambient, emissive, diffuse, shininess, specular, texture-index

Default Value None

matrix

Describes a coordinate frame of the **transform** node.

```
(matrix
  (
    ( R11 R12 R13 Tx )
    ( R21 R22 R33 Ty )
    ( R31 R32 R34 Tz )
    ( 0 0 0 1 )
  )
)
```

Parent nodes transform

Number of occurrences per parent node 1

Required member nodes list

Optional member nodes None

Default Value The default value of the matrix is the identity matrix:

```
( 1 0 0 0 )
( 0 1 0 0 )
( 0 0 1 0 )
( 0 0 0 1 )
```

mesh

Contains vertex, normal, color-vertex, uv-coord, and indexed face set information. Currently there are two face sets: triangle face set and indexed face set. A face set is required for the vertex list, if there are no other face sets it is assumed that the other geometric properties are per vertex. So, if there are no nrm-fs nodes then we are assuming that there is a normal per vertex.

It is possible to have two face set definitions for each geometric type in the node; this is for the case where we have a stripped model, we want to keep the triangle faceset as well.

```
(mesh "identifier"  
  (vertex)  
  (normals)  
  (uvs)  
  (tri-fs)  
  (color-fs)  
  (tex-fs)  
  (nrm-fs)  
  (tri-fs)  
)
```

Parent nodes geometry

Number of occurrences per parent node 1

Required member nodes for a triangle face set tri-fs, vertex

Required member nodes for an indexed face set tri-fx, vertex, uvs

Optional member nodes color-fs, normals, nrm-fs, tex-fs

Default Value None

node-relations

Coordinate frame offset to apply to animation for determining current animation transformation set.

```
(node-relations
  (
    ("transform_identifier1"
      (pos ( x y z)
        (quat ( a b c d e )
          )
      )
    ("transform_identifier2"
      (pos ( x y z)
        (quat ( a b c d e )
          )
      )
    ...
  )
)
```

Parent nodes child-model

Number of occurrences per parent node 1

Required member nodes A collection of list nodes. Each list node in the collection contains the identifier of a **transform** in the hierarchy and **pos** and **quat** nodes.

Optional member nodes None

Default Value None

normals

Contains a collection of list nodes defining the normals for the **mesh**'s vertices.

```
(normals
  (
    ( x y z )
    ( x y z )
    ...
  )
)
```

Parent nodes mesh

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes 0 or more list nodes containing normal information.

Default Value None

nrm-fs

Contains a list node describing face sets for the **mesh**'s normals.

This node is required in the **mesh** if the number of list nodes under **normals** does not equal the number of list nodes under the **vertex** node.

```
(nrm-fs
  ( normal1_facesetdata1 normal1_facesetdata2 normal1_facesetdata3 normal2_facesetdata1
    normal2_facesetdata2 normal2_facesetdata3 ... )
)
```

Parent nodes mesh

Number of occurrences per parent node 0 or 1

Required member nodes A list node of face set data

Optional member nodes None

Default Value None

on-load-cmds

Contains a list of commands and modifications that are applied to the model database after all the data has been loaded.

```
(on-load-cmds
  (
    (add-deformer)
    (add-sockets)
    (set-global-radius)
    (add-childmodels)
    (anim-weightsets)
    (anim-bindings)
    (set-node-flags)
    (set-repl-lod-original)
  )
)
```

Parent nodes lt-model-0

Number of occurrences per parent node 0 or more

Required member nodes None

Optional member nodes add-child-models, add-deformer, add-sockets, anim-bindings, anim-weightsets, set-global-radius, set-node-flags, set-repl-lod-original

Default Value None

orientation

Sets the orientation for an OBB.

(orientation (x y z w))

Parent nodes add-node-obb

Number of occurrences per parent node 0 or 1

Required member nodes Values defining a quaternion representing a rotation.

Optional member nodes None

Default Value 0 0 0 1

parent

Identifies the grouping node for associated node.

(parent "parent_node_identifier")

Parent nodes shape, socket

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes None

Default Value None

piece-priorities

Sets a shape's collapse priority for lod generation. Shape with a larger priority value are collapsed last.

```
(piece-priorities
  (
    ("shape_identifier" priority)
    ...
  )
)
```

Parent nodes set-repl-lod-original

Number of occurrences per parent node 0 or 1

Required member nodes A collection of list nodes containing a **shape** node identifier and the collapse priority for that shape.

Optional member nodes None

Default Value None

pos

Sets the position of a **socket** in relation to the parent **transform**.

```
(pos  
  ( x y z )  
)
```

Parent nodes add-node-obb, socket

Number of occurrences per parent node 1

Required member nodes A list node containing the position coordinates.

Optional member nodes None

Default Value 0 0 0

posquat

Modifies the position and quaternion values of a **transform**.

```
(posquat  
  (  
    (pos) (quat)  
  )  
)
```

Parent nodes frames

Number of occurrences per parent node 1

Required member nodes A collection of **pos** and **quat** nodes

Optional member nodes None

Default Value None

quat

Sets the quaternion orientation for a **socket**.

```
(quat  
  ( a b c d )  
)
```

Parent nodes socket

Number of occurrences per parent node 1

Required member nodes A list node containing the quaternion orientation.

Optional member nodes None

Default Value None

render-priority

Sets the rendering priority for the shape.

(render-priority number)

Parent nodes shape

Number of occurrences per parent node 1

Required member nodes A numerical value between 0 and 128 that sets the order in which shapes are rendered.
Lower numbers are rendered first.

Optional member nodes None

Default Value None

set-global-radius

This value is used to determin a model's visibility.

(set-global-radius *radius_value*)

Parent nodes on-load-cmds

Number of occurrences per parent node 0 or 1

Required member nodes None

Optional member nodes None

Default Value 96

set-node-flags

Sets flags on a transform node.

The *transform_identifier* indicates the transform node to which the flag will be applied.

The flag may be either 0 or 2.

0 No flags.

2 Positional information is ignored for animation updates.

```
(set-node-flags
  (
    ("transform_identifier" flag)
    ...
  )
)
```

Parent nodes on-load-cmds

Number of occurrences per parent node 0 or 1

Required member nodes A collection of nodes identifying **transform** nodes and the flags to apply to each.

Optional member nodes None

Default Value The value for *flag* defaults to 0.

set-repl-lod-original

Describes the parameters required to generate level of detail data from the model database.

```
(set-repl-lod-original  
  (piece-priorities)  
  (dists)  
  (tri-%)  
)
```

Parent nodes on-load-cmds

Number of occurrences per parent node 0 or 1

Required member nodes dists, tri-%

Optional member nodes piece-priorities

Default Value None

shape

A description of the model's geometrical data and the appearance of that geometry.

```
(shape "identifier"  
  (parent)  
  (geometry)  
  (appearance)  
)
```

Parent nodes It-model-0

Number of occurrences per parent node One or more

Required member nodes appearance, geometry, parent

Optional member nodes None

Default Value None

size

Sets the size of the OBB.

(size (x y z))

Parent nodes add-node-obbb

Number of occurrences per parent node 0 or 1

Required member nodes Set of values defining the dimensions of the OBB.

Optional member nodes None

Default Value 1 1 1

shininess

Sets the shininess of the material.

Higher shininess values shrink the size of reflected light on the shape. A value of 0 indicates that shininess is off.

(shininess *value*)

Parent nodes material

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes None

Default Value 0

skel-deformer

Defines the parameters for skeletal deformation of a model. The deformer is associated with a **shape** identified with the **target** member node. The influences have to exist in the context the deformer is created.

There can be one **skel-deformer** associated with each **shape**.

```
(skel-deformer "identifier"  
  (target)  
  (influences)  
  (weightsets)  
)
```

Parent nodes add-deformer

Number of occurrences per parent node 0 or 1

Required member nodes influences, target, weightset

Optional member nodes None

Default Value None

socket

Describes an extension to the node hierarchy where other models are attached.

```
(socket "identifier"
  (parent)
  (pos)
  (quat)
)
```

Parent nodes add-sockets

Number of occurrences per parent node 0 or more

Required member nodes parent

Optional member nodes pos, quat

Default Value None

specular

Defines the color of reflected light on the material.

```
(specular  
  ( r g b )  
)
```

Parent nodes material

Number of occurrences per parent node 0 or 1

Required member nodes A list node containing the rgb values of the color.

Optional member nodes None

Default Value 0

target

Identifies the node to which this node's parent node will apply.

(target *"target_identifier"*)

Parent nodes anim, skel-deformer

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes None

Default Value None

texture-index

Sets the reference number for the texture to use. The reference number is the index into the texture map list.

(texture-index *reference_number*)

Parent nodes material

Number of occurrences per parent node 0 or 1

Required member nodes None

Optional member nodes None

Default Value 0

tex-fs

Contains a list of texture coordinate indices into the texture list.

```
(tex-fs  
  ( a b c d e f g h i )  
)
```

Parent nodes mesh

Number of occurrences per parent node 0 or 1

Required member nodes A list node containing texture indices into the texture list.

Optional member nodes None

Default Value None

texture-bindings

Describes which textures to load for the model. The index is the texture number specified in the renderstyle info for a model piece. The file name identifies the file used for the texture when rendering. There can be only one **texture-bindings** node per tools-info.

```
(texture-bindings
  (
    (index_value "filename")
  )
)
```

Parent nodes tools-info

Number of occurrences per parent node 1

Required member nodes list node with index value and filename

Optional member nodes None

Default Value None

tools-info

Contains information that tools need but are not specifically germane to the engine. The **tools-info** node has a texture bindings list for the model that contains the node.

```
(tools-info
  (
    (texture-bindings)
  )
)
```

Parent nodes lt-model-0

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes texture-binding

Default Value None

transform

A transform is a grouping node that defines a coordinate system for its children.
A **shape** listed under a **transform**'s **children** node is considered rigid geometry.

```
(transform "identifier"  
  (matrix)  
  (coord-frame-type)  
  (children)  
)
```

Here is an example of a transform with an embedded shape:

```
(transform "identifier"  
  (scale (x y z))  
  (pos (x y z))  
  (children  
    (  
      (shape "identifier"  
        (geometry)  
      )  
    )  
  )  
)
```

Parent nodes children

Number of occurrences per parent node 0 or more

Required member nodes None

Optional member nodes children, coord-frame-type, matrix

Default Value None

translation

Positional offset added to a root node of an animation.

This value is applied to an animation in world space.

```
(translation  
  ( x-offset y-offset z-offset )  
)
```

Parent nodes anim-binding

Number of occurrences per parent node 0 or 1

Required member nodes A list node containing the x, y, and z offsets.

Optional member nodes None

Default Value (0 0 0)

tri-%

A list that specifies the percentage of triangles to retain for each level of detail. This applies all the shapes in the model database.

Each value in the **tri-%** list node is associated with a value in the **dist** list node. Therefore, the number of values here must equal the number of values in the **dist** node.

```
(tri-%  
  (percentage1 percentage2 percentage3)  
)
```

Parent nodes set-repl-lod-original

Number of occurrences per parent node 1

Required member nodes A list node containing decreasing values.

Optional member nodes None

Default Value None

tri-fs

Defines a list of triangle indices into the vertex list.

These values are also used for the normal and texture lists if there are no **nrm-fs** and **tex-fs** nodes (face sets).

```
(tri-fs
  ( a b c d e f g a g )
)
```

Parent nodes mesh

Number of occurrences per parent node 1

Required member nodes A list node containing triangle indices in groups of three.

Optional member nodes None

Default Value None

uvs

Defines a list of vectors that represent the u-v coordinates of the texture. Values should be normalized float values.

```
(uvs
  (
    ( u v )
    ( u v )
    ...
  )
)
```

Parent nodes mesh

Number of occurrences per parent node 0 or 1

Required member nodes A collection of list nodes each containing uv coordinates.

Optional member nodes None

Default Value None

vertex

Contains a list of vectors representing the vertices of the model.

```
(vertex
  (
    ( x y z )
    ( x y z )
    ...
  )
)
```

Parent nodes mesh

Number of occurrences per parent node 1

Required member nodes A collection of list nodes each containing vertex x, y, and z coordinates.

Optional member nodes None

Default Value None

weightsets

Sets the vertex weight for each influence in the **skel-deformer**.

There is a list for every **vertex** or control point in the **shape's geometry**. The *influence_index* is an index into the **influences** list of the **skel-deformer**. Index 0 is the first element in the **influences** list node. The *weight* element is how much the transform influences that vertex.

The number of items in the **weightsets** list must equal the number of vertices in the node identified by the **skel-deformer's target** node.

```
(weightsets
  (
    (influence_index weight influence_index weight)
    ...
  )
)
```

Parent nodes skel-deformer

Number of occurrences per parent node 1

Required member nodes A collection of list nodes containing influence-weight pairs.

Optional member nodes None

Default Value None

weight-set

Informs **anim-binding** which **anim-weightset** to use.

(weight-set *"identifier"*)

Parent nodes anim-binding

Number of occurrences per parent node 0 or 1

Required member nodes None

Optional member nodes None

Default Value None

weights

Contains a list of animation blending weights for each transform in the hierarchy.

```
(weights  
  (weight1 weight2 ...)  
)
```

Parent nodes anim-weightsets

Number of occurrences per parent node 1

Required member nodes A list of weights, one for each transform in the hierarchy.

Optional member nodes None

Default Value None

World Schema .LTA Reference

This reference describes the nodes used for the LithTech world schema. A world schema includes all information necessary to render world geometry.

The nodes are presented in alphabetical order.

World Schema .LTA Node Hierarchy

The following structure lists all the various nodes in the world schema. This structure only lists typed nodes, see the individual node references for member node information.

```
(world
  (header
    (versioncode)
    (infostring)
  )
  (polyhedronlist
    (
      (polyhedron
        (
          (color)
          (pointlist)
          (polylist
            (
              (editpoly
                (f)
                (n)
                (dist)
                (textureinfo
                  ()
                  ()
                  ()
                  (sticktopoly)
                  (name)
                )
              )
              (flags)
              (shade)
            )
          )
        )
      )
    )
  )
  (nodehierarchy
    (worldnode
      (type)
      (label)
      (nodeid)
      (flags)
      (properties
        (name)
        (propid)
      )
      (childlist
        (
          (worldnode)
          (worldnode)
        )
      )
    )
  )
  (globalproplist
    (proplist
      (
        (string)
        (vector)
        (color)
      )
    )
  )
)
```

```
(real)
(flags)
(bool)
(logint)
(rotation)
)
)
)
(navigatorposlist
  (navigatorpositem
    (navigator
      (up)
      (forward)
      (right)
      (position)
      (lookat)
      (lookatdist)
    )
  )
)
)
```

World Schema Example

Descriptions and comments in **bold**.

```
(world
  ( header
    (
      ( versioncode 1 )
      Version number hint
      ( infostring "AmbientLight 60 80 80" )
      World info string you specify in DEdit
    )
  )
  ( polyhedronlist
    (
      ( polyhedron
        (
          ( color 255 255 255 )
          RGB color of the polyhedron
          ( pointlist
            Vertex list used by the polyhedron
            ( 16132.921875 2143.031006 7773.587891 )
            ( 17147.556641 2288.290771 5322.891113 )
            ( 17528.560547 786.472412 5101.568848 )
          )
          ( polylist
            Polygons in the polyhedron
            (
              ( editpoly
                ( f 2 1 0 )
                Indexes that make up the polygon
                ( n 0.906324 0.173118 0.385496 )
                Normal for the polygon
                ( dist 17989.337891 )
                This value is re-calculated by DEdit when it saves the .LTA file,important for processor.exe but ignored by DEdit
                ( textureinfo
                  ( 0.000000 274.472412 493.568848 )
                  O origin
                  ( 0.000000 0.000000 0.125000 )
                  P value
                  ( 0.000000 -0.125000 0.000000 )
                  Q value
                  ( sticktopoly 1 )
                  This is set in DEdit
                  ( name "Textures\Sample\Tr0141.dtx" )
                  Texture name
                )
                ( shade 0 0 0 )
                ( physicsmaterial "Physics\ice.lta" )
                Physics material name
              )
            )
          )
        )
      )
    )
  )
  ( nodehierarchy
    ( worldnode
```

```

( type null )
Null | brush | object
( label "Container" )
( nodeid 1735 )
This is generated by DEdit when it saves the .LTA file,
ignored when read by DEdit
( flags
  ( worldroot expanded )
  selected | hidden | path | worldroot | expanded
)
( properties
  ( propid 0 )
  This is an index to a proplist in the globalproplist below
)
( childlist
  (
    ( worldnode
      ( type brush )
      ( brushindex 0 )
      ( nodeid 1736 )
      ( flags ( ) )
      ( properties
        ( name "Brush" )
        ( propid 1 )
      )
    )
  )
)
)
)
)
)
( globalproplist
  (
    ( proplist
      (
        ( string "Name" ( )
          ( data "Brush0")
          <type> <name> (<flags>) (<data>)
        )
        ( vector "Pos" ( )
          ( data
            ( vector
              (0.000000 0.000000 0.000000)
            )
          )
        )
        ( rotation "Rotation" ( )
          ( data
            (eulerangles
              (0.000000 0.000000 0.000000)
            )
          )
        )
        ( bool "Solid" ( )
          ( data 1 )
        )
        ( bool "Nonexistant" ( )
          ( data 0 )
        )
        ( bool "Invisible" ( )
          ( data 0 )
        )
        ( bool "Translucent" ( )

```

```
( data 0 )
)
( bool "SkyPortal" ( )
  ( data 0 )
)
( bool "FullyBright" ( )
  ( data 0 )
)
( bool "FlatShade" ( )
  ( data 0 )
)
( bool "GouraudShade" ( )
  ( data 1 )
)
( bool "LightMap" ( )
  ( data 1 )
)
( bool "Subdivide" ( )
  ( data 1 )
)
( bool "HullMaker" ( )
  ( data 0 )
)
( bool "AlwaysLightMap" ( )
  ( data 0 )
)
( bool "DirectionalLight" ( )
  ( data 0 )
)
( bool "Portal" ( )
  ( data 0 )
)
( bool "NoSnap" ( )
  ( data 0 )
)
( bool "SkyPan" ( )
  ( data 0 )
)
( bool "Additive" ( )
  ( data 0 )
)
( bool "TerrainOccluder" ( )
  ( data 0 )
)
( bool "TimeOfDay" ( )
  ( data 0 )
)
( bool "VisBlocker" ( )
  ( data 0 )
)
( bool "NotAStep" ( )
  ( data 0 )
)
( longint "DetailLevel" ( )
  ( data 0.000000 )
)
( string "Effect" ( ) )
( string "EffectParam" ( ) )
( real "FrictionCoefficient" ( )
  ( data 1.000000 )
)
)
```

```

    )
  )
)
( navigatorposlist
(
  ( navigatorpositem
    ( navigator
      Camera location
      ( up 0.000000 0.707107 0.707107 )
      ( forward 0.000000 -0.707107 0.707107 )
      ( right 1.000000 0.000000 0.000000 )
      ( position 0.000000 200.100006 -200.100006 )
      ( lookout 0.000000 0.000000 0.000000 )
      ( lookoutdist 282.984131 )
    )
    ( navigator
      ( up 0.000000 1.000000 0.000000 )
      ( forward 1.000000 0.000000 0.000000 )
      ( right 0.000000 0.000000 -1.000000 )
      ( position -6144.000000 0.000000 0.000000 )
      ( lookout 0.000000 0.000000 0.000000 )
      ( lookoutdist -0.001327 )
    )
    ( navigator
      ( up 0.000000 0.000000 1.000000 )
      ( forward 0.000000 -1.000000 0.000000 )
      ( right 1.000000 0.000000 0.000000 )
      ( position 0.000000 6144.000000 0.000000 )
      ( lookout 0.000000 0.000000 0.000000 )
      ( lookoutdist -0.001327 )
    )
    ( navigator
      ( up 0.000000 1.000000 0.000000 )
      ( forward 0.000000 0.000000 1.000000 )
      ( right 1.000000 0.000000 0.000000 )
      ( position 0.000000 0.000000 -6144.000000 )
      ( lookout 0.000000 0.000000 0.000000 )
      ( lookoutdist -0.001327 )
    )
    ( description "POS(100.0, 200.1, -200.1)" )
    String description
  )
)
)
)
)

```

bool

Sets a Boolean property.

The *type* identifies the data. The *flags* set the flags for the type.

```
(bool "type"  
  (flags)  
  (data)  
)
```

Parent nodes proplist

Number of occurrences per parent node 0 or more

Required member nodes data

Optional member nodes A node containing the *flags*.

Default Value None

childlist

Contains a collection of **worldnode** nodes.

```
(childlist
  (
    (worldnode)
    (worldnode)
  )
)
```

Parent nodes worldnode

Number of occurrences per parent node 1

Required member nodes None.

Optional member nodes A collection of 0 or more **worldnode** nodes.

Default Value None

color

Sets the RGB color of the **polyhedron**.

(color *red_value green_value blue_value*)

Parent nodes polyhedron

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes None

Default Value None

color

Sets a color property.

The *type* identifies the data. The *flags* set the flags for the type.

```
(bool "type"  
  (flags)  
  (data)  
)
```

Parent nodes proplist

Number of occurrences per parent node 0 or more

Required member nodes data

Optional member nodes A node containing the *flags*.

Default Value None

data

Sets the data for a property.

The value will depend on the parent node.

(data *value*)

Parent nodes bool, color, flags, longint, real, rotation, string, vector

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes None

Default Value None

description

Sets a description for a navigatorpositem node.

(description *"description_text"*)

Parent nodes navigatorpositem

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes None

Default Value None

dist

This value is re-calculated by Dedit when it saves the LTA file, important for processor.exe but ignored by DEdit. Together with the **n** node this node defines the plane of the polygon.

(dist *value*)

Parent nodes editpoly

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes None

Default Value None

editpoly

Sets polygon vertices and other data.

```
(editpoly
  (f)
  (n)
  (dist)
  (textureinfo)
  (flags)
  (shade)
)
```

Parent nodes editpoly

Number of occurrences per parent node 1 or more

Required member nodes dist, f,

Optional member nodes None

Default Value None

f

Sets vertices to be modified by the **editpoly**. The values indicate the indices into the **pointlist** node.

(f *index1 index2 index3 ...*)

Parent nodes editpoly

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes None

Default Value None

flags

Sets the flags for the **proplist** or **worldnode**.

Possible values for the **proplist** node include a combination of one or more of the following:

hidden
radius
dims
fieldofview
groupowner
group1
group2
group3
group4
group5
group6
fovradius
objectlink
filename
bezierprevtangent
beziernexttangent
staticlist

dynamiclist

Possible values for the **worldnode** node include

selected
hidden
path
worldroot
expanded

(flags *combination_of_flags*)

Parent nodes editpoly, proplist, worldnode

Number of occurrences per parent node 0 or 1

Required member nodes None

Optional member nodes None

Default Value No flags set.

forward

Sets the forward vector coordinates for the **navigator** node.

(forward *x_coord y_coord z_coord*)

Parent nodes navigator

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes None

Default Value None

header

Sets header information for the world schema.

```
(header  
  (versioncode)  
  (infostring)  
)
```

Parent nodes world

Number of occurrences per parent node One

Required member nodes versioncode

Optional member nodes infostring

Default Value None

infostring

Sets the Worldinfo string specified in DEdit.

(infostring *"string"*)

Parent nodes header

Number of occurrences per parent node 0 or 1

Required member nodes None

Optional member nodes None

Default Value None

label

Sets the identifier for the **world** node.

(label *"identifier"*)

Parent nodes worldnode

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes None

Default Value None

longint

Sets a long integer property.

The *type* identifies the data. The *flags* set the flags for the type.

```
( longint "type"  
  ( flags )  
  ( data )  
)
```

Parent nodes proplist

Number of occurrences per parent node 0 or more

Required member nodes data

Optional member nodes A node containing the *flags*.

Default Value None

lookat

Sets the lookat position for the **navigator** node.

```
(lookat x_coord y_coord z_coord )
```

Parent nodes navigator

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes None

Default Value None

lookatdist

Sets the lookat distance for the **navigator** node.

(lookatdist *distance*)

Parent nodes navigator

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes None

Default Value None

n

Sets the normal for the polygon. The x, y, and z coordinates are relative to the polygon. Together with the **dist** node this node defines the plane of the polygon.

(n *x_coord y_coord z_coord*)

Parent nodes editpoly

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes None

Default Value None

name

Sets the path to the texture file from the project directory.

(name *"path"*)

Parent nodes textureinfo

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes None

Default Value None

nodeid

Sets the DEdit identifier for the **world** node.

(nodeid *identifier*)

Parent nodes worldnode

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes None

Default Value None

navigator

Defines one of the four views of a bookmark in DEdit.

The up, forward, and right nodes set the orthonormal orientation for the **navigator** node.

```
(navigator
  (up)
  (forward)
  (right)
  (position)
  (lookat)
  (lookatdist)
)
```

Parent nodes navigatorposition

Number of occurrences per parent node 4 (the DEdit tool has four views)

Required member nodes forward, lookat, lookatdist, position, right, up

Optional member nodes None

Default Value None

navigatorpositem

Defines a DEDit bookmark.

```
(navigatorpositem
  (navigator)
  (navigator)
  (navigator)
  (navigator)
  (description)
)
```

Parent nodes navigatorposlist

Number of occurrences per parent node 1

Required member nodes Four **navigator** nodes, one **description** node

Optional member nodes None

Default Value None

navigatorposlist

Contains a collection of DEdit camera positions.

```
(navigatorposlist
  (
    (navigatorpositem)
    ...
  )
)
```

Parent nodes world

Number of occurrences per parent node 0 or 1

Required member nodes None

Optional member nodes A collection of 0 or more **navigatorpositem** nodes

Default Value None

physicsmaterial

Sets the path to the physics material file from the project directory.

(physicsmaterial *"path"*)

Parent nodes editpoly

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes None

Default Value None

pointlist

Contains a list of vertices used by the **polyhedron** node.
The list is 0-based.

```
(pointlist
  ( x_coord y_coord z_coord )
  ...
)
```

Parent nodes polyhedron

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes Any number of list nodes containing x, y, and z coordinates

Default Value None

polyhedron

Describes the geometry of a non-model object in the world.

```
(polyhedron
  (
    (color)
    (pointlist)
    (polylist)
  )
)
```

Parent nodes polyhedronlist

Number of occurrences per parent node 1 or more

Required member nodes color, pointlist, polylist

Optional member nodes None

Default Value None

polyhedronlist

Contains a collection of **polyhedron** nodes.

```
(polyhedronlist
  (
    (polyhedron)
    ...
  )
)
```

Parent nodes world

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes A collection of one or more **polyhedron** nodes.

Default Value None

polylist

Contains a collection of **editpoly** nodes.

```
(polylist
  (
    (editpoly)
    ...
  )
)
```

Parent nodes polyhedron

Number of occurrences per parent node 1

Required member nodes A collection of one or more **editpoly** nodes.

Optional member nodes None

Default Value None

position

Sets the position of the **navigator** node.

(position *x_coord* *y_coord* *z_coord*)

Parent nodes navigator

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes None

Default Value None

properties

Contains a **propid** node.

```
(properties  
  (propid)  
)
```

Parent nodes worldnode

Number of occurrences per parent node 1

Required member nodes propid

Optional member nodes None

Default Value None

propid

Identifies the **proplist** to use for this **worldnode**.
The *index* is an index into the **proplist**.

(propid *index*)

Parent nodes properties

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes None

Default Value None

proplist

Defines a set of properties that can be applied to a **worldnode** using **propid**.

```
(proplist
  (
    (string)
    (vector)
    (color)
    (real)
    (flags)
    (bool)
    (longint)
    (rotation)
  )
)
```

Parent nodes globalproplist

Number of occurrences per parent node 1 or more

Required member nodes None

Optional member nodes bool, color, flags, longint, real, rotation, string, vector

Default Value None

real

Sets a real property.

The *type* identifies the data. The *flags* set the flags for the type.

```
(real "type"  
  (flags)  
  (data)  
)
```

Parent nodes proplist

Number of occurrences per parent node 0 or more

Required member nodes data

Optional member nodes A node containing the *flags*

Default Value None

right

Sets the right vector coordinates for the **navigator** node.

(right *x_coord y_coord z_coord*)

Parent nodes navigator

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes None

Default Value None

rotation

Sets a rotation property.

The *type* identifies the data. The *flags* set the flags for the type.

```
(rotation "type"  
  (flags)  
  (data)  
)
```

Parent nodes proplist

Number of occurrences per parent node 0 or more

Required member nodes data

Optional member nodes A node containing the *flags*.

Default Value None

shade

Sets the shade (as an rgb value) of the **editpoly**.

(shade *r g b*)

Parent nodes editpoly

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes None

Default Value None

sticktopoly

Locks this texture onto the polygon.

If *setting* is 1, the texture the texture is locked onto the polygon. If *setting* is 0, the texture is not locked.

(sticktopoly *setting*)

Parent nodes textureinfo

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes None

Default Value None

string

Sets a string property.

The *type* identifies the data. The *flags* set the flags for the type.

```
(string "type"  
  (flags)  
  (data)  
)
```

Parent nodes proplist

Number of occurrences per parent node 0 or more

Required member nodes data

Optional member nodes A node containing the *flags*.

Default Value None

textureinfo

Sets texture information for the **editpoly**.

The first three member nodes set the o, p, and q values for the texture.

```
(textureinfo
  (o_value)
  (p_value)
  (q_value)
  (sticktopoly)
  (name)
)
```

Parent nodes editpoly

Number of occurrences per parent node 1

Required member nodes o, p, and q value nodes, **sticktopoly**, **name**

Optional member nodes None

Default Value None

type

Sets the type of the **worldnode**.

The *type* can be **null**, **brush**, or **object**.

(type *type*)

Parent nodes worldnode

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes None

Default Value None

up

Sets the up vector coordinates for the **navigator** node.

(up *x_coord* *y_coord* *z_coord*)

Parent nodes navigator

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes None

Default Value None

vector

Sets a vector property.

The *type* identifies the data. The *flags* set the flags for the type.

```
(vector "type"  
  (flags)  
  (data)  
)
```

Parent nodes proplist

Number of occurrences per parent node 0 or more

Required member nodes data

Optional member nodes A node containing the *flags*.

Default Value None

versioncode

Sets the version code for the world schema.

(versioncode *version*)

Parent nodes header

Number of occurrences per parent node 1

Required member nodes None

Optional member nodes None

Default Value None

world

Defines the information for a world schema.

```
(world
  (header)
  (polyhedronlist)
  (nodehierarchy)
  (globalproplist)
  (navigatorposlist)
)
```

Parent nodes none

Number of occurrences per parent node One per file

Required member nodes globalproplist, header, navigatorposlist, nodehierarchy, polyhedronlist

Optional member nodes None

Default Value None

worldnode

Sets information for a **worldnode**.

```
(worldnode
  (type)
  (label)
  (nodeid)
  (flags)
  (properties)
  (childlist)
)
```

Parent nodes hierarchy, childlist

Number of occurrences per parent node 1

Required member nodes flags, label, nodeid, properties, type

Optional member nodes childlist

Default Value None